

Architecture.md v1.1

Chapter 8 — Event System & Workflow Engine

This chapter defines how AI OS executes workflows, tracks system events, and coordinates complex multi-agent operations.

The Event System enables automation, observability, and future asynchronous execution without tightly coupling components.

8.1 Purpose

The Event System shall:

- Record significant system activities.
- Coordinate workflow execution.
- Trigger automation rules.
- Enable auditing and debugging.
- Support future background processing.

The Event System does **not** perform business logic.

8.2 Event Philosophy

Every meaningful action inside AI OS is represented as an **Event**.

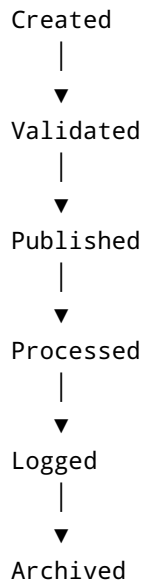
Examples:

- User Request Received
- Router Decision
- Agent Started
- Agent Completed
- Memory Updated
- Approval Requested
- Workflow Finished
- Error Detected

Events describe **what happened**, not **what should happen**.

8.3 Event Lifecycle

Every event follows the same lifecycle.



Events are immutable after publication.

8.4 Standard Event Structure

Every event shall contain:

- Event ID (UUID)
- Event Type
- Timestamp
- Source Component
- Related Request ID
- Status
- Metadata
- Severity Level

Example:

```
Event ID: EVT-000452
Type: AGENT_COMPLETED
Source: Analytics Agent
Request: REQ-000091
```

Timestamp: 2026-07-02T18:10:00Z
Status: SUCCESS

8.5 Event Types

The following event categories are reserved.

System Events

- SYSTEM_STARTED
 - SYSTEM_STOPPED
 - CONFIG_UPDATED
-

Request Events

- REQUEST_RECEIVED
 - REQUEST_ROUTED
 - REQUEST_COMPLETED
-

Agent Events

- AGENT_REGISTERED
 - AGENT_STARTED
 - AGENT_COMPLETED
 - AGENT_FAILED
-

Memory Events

- MEMORY_READ
 - MEMORY_UPDATED
 - MEMORY_DELETED
 - MEMORY_ARCHIVED
-

Security Events

- PERMISSION_GRANTED
- PERMISSION_DENIED
- APPROVAL_REQUESTED
- APPROVAL_RECEIVED

Workflow Events

- WORKFLOW_STARTED
 - WORKFLOW_COMPLETED
 - WORKFLOW_CANCELLED
-

8.6 Workflow Engine

The Workflow Engine is responsible for executing multi-step tasks.

Responsibilities:

- Build execution pipelines.
- Maintain execution state.
- Pass context between stages.
- Handle failures.
- Return final result.

It works alongside the Agent Manager but does not replace it.

8.7 Workflow Types

Sequential Workflow (v0.1)

Tasks execute one after another.

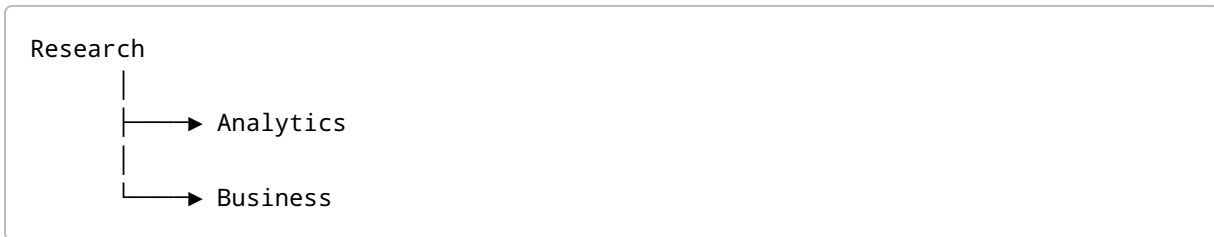
Example:



Parallel Workflow (Future)

Independent tasks execute simultaneously.

Example:



Conditional Workflow (Future)

Next step depends on previous output.

Example:

If research confidence > 80%

↓

Generate report

Else

↓

Ask for clarification

8.8 Workflow State

Every workflow maintains one state.

Possible states:

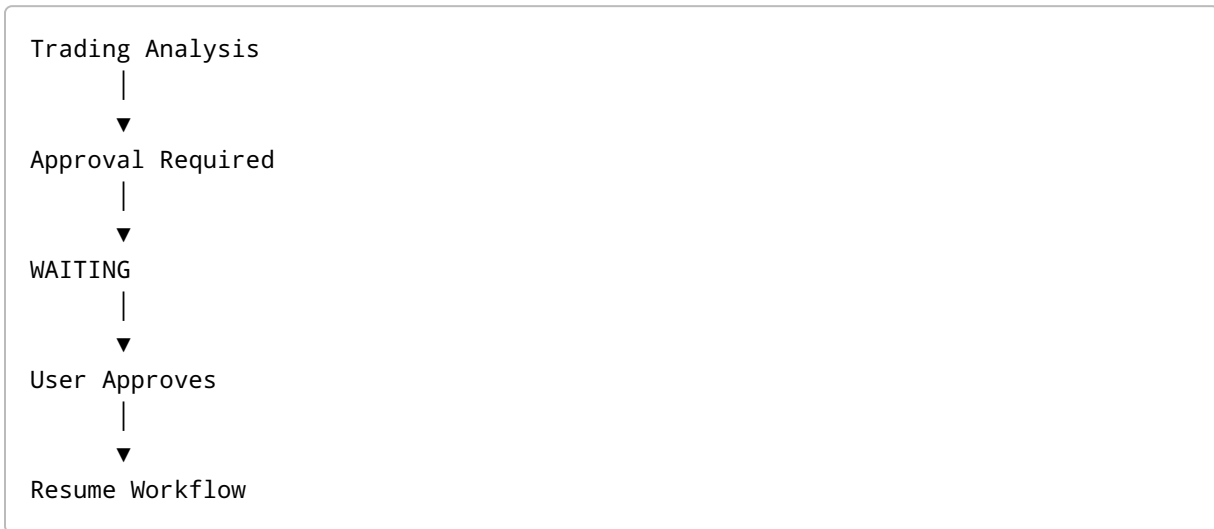
- CREATED
- WAITING
- RUNNING
- PAUSED
- COMPLETED
- FAILED
- CANCELLED

Only the Workflow Engine may change workflow state.

8.9 Human Approval Integration

Workflows requiring user approval shall pause execution.

Example:



Timeouts automatically transition the workflow to **CANCELLED**.

8.10 Error Handling

If one workflow step fails:

1. Log the failure.
2. Generate an event.
3. Notify the Agent Manager.
4. Decide whether to:
 5. Retry
 6. Skip
 7. Abort

The Workflow Engine never crashes the entire system.

8.11 Event Storage

Events are stored separately from user memory.

Purpose:

- Debugging
- Monitoring
- Performance analysis
- Audit history

Events are **not** treated as long-term user memories.

8.12 Automation Triggers

Future versions may define automation rules.

Example:

```
Event :  
REQUEST_COMPLETED  
  
↓  
  
Condition:  
Task = Instagram Analytics  
  
↓  
  
Action:  
Generate Daily Report
```

Automation is driven by events rather than polling.

8.13 Event Subscribers

Components may subscribe to event categories.

Examples:

Logging Service

Subscribes to:

- All events
-

Notification Service (Future)

Subscribes to:

- Approval Requested
 - Workflow Completed
-

Analytics Service (Future)

Subscribes to:

- Performance Events

Subscribers never modify published events.

8.14 Event Priorities

Each event has a priority.

Critical

- Security violations
 - Database failures
 - Memory corruption
-

High

- Workflow failures
 - Permission denials
-

Normal

- Agent execution
 - Request completion
-

Low

- Debug messages
 - Health checks
-

8.15 Workflow Design Principles

Every workflow shall be:

- Deterministic
- Observable
- Restartable
- Traceable
- Independent
- Recoverable

No workflow may rely on undocumented behavior.

8.16 Event System Rules

The Event System shall never:

- Execute business logic.
- Modify memory.
- Call worker agents directly.
- Make routing decisions.

Its role is to observe, coordinate, and notify.

8.17 Future Expansion

Future releases may introduce:

- Distributed event bus
- Scheduled workflows
- Cron-based automation
- Event replay
- Event analytics dashboard
- Real-time notifications
- Workflow templates
- Visual workflow editor

These enhancements must preserve backward compatibility with the core event architecture.

End of Chapter 8

Next Chapter: Chapter 9 — Security, Permissions & Human Approval System